Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche dans un tableau linéaire, analyse de complexité et arbitrage algorithmique

Synthèse exécutive : Ce cours traite des deux stratégies fondamentales de recherche d'une donnée au sein d'une structure linéaire (tableau) : la recherche séquentielle (linéaire) et la recherche dichotomique (fondée sur le paradigme 'diviser pour régner'). L'enjeu central repose sur la relation directe entre l'organisation des données (tableau trié ou non trié) et la performance temporelle de l'algorithme. La recherche séquentielle parcourt le tableau de manière exhaustive en O(n) sans aucune précondition sur les données. À l'inverse, la recherche dichotomique divise par deux l'espace de recherche à chaque étape, garantissant une complexité logarithmique en O(log n), à la condition impérative que le tableau soit préalablement trié. Le cours détaille la manipulation dynamique des pointeurs d'indices (début, fin, milieu), l'analyse des complexités au pire des cas, ainsi que l'arbitrage économique entre le coût d'un tri initial et la fréquence des requêtes de recherche.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est l'algorithme de recherche le plus intuitivement naturel et universel. L'algorithme initialise un pointeur d'indice à 0 et parcourt chaque case du tableau une par une à l'aide d'une boucle. À chaque itération, une comparaison logique est effectuée entre la valeur contenue dans la case courante et la valeur recherchée (la cible). Deux issues sont possibles : 1. Succès : La valeur recherchée correspond à l'élément courant. L'algorithme s'interrompt immédiatement et renvoie l'indice de la case. 2. Échec : L'intégralité du tableau a été parcourue jusqu'à l'indice n-1 sans trouver la cible. L'algorithme renvoie une valeur conventionnelle signifiant l'absence (généralement -1). Dans le pire des cas, si le tableau contient n éléments et que la cible se trouve à l'indice n-1 ou est absente, l'algorithme doit réaliser exactement n comparaisons. Sa complexité temporelle est donc notée O(n). Son principal avantage stratégique est qu'il ne nécessite aucun pré-traitement ni organisation préalable des données.

- Parcours exhaustif et séquentiel du tableau du premier indice (0) au dernier indice (n-1).
- Absence totale de précondition : utilisable indifféremment sur un tableau trié ou non trié.
- Complexité temporelle linéaire O(n) au pire des cas (élément situé à la toute fin ou absent).
- Arrêt précoce dès que l'élément cible est identifié dans la structure.

Cas concret enseignant : *Analogie du professeur : La recherche d'une clé tombée dans un tas de feuilles mortes. Le tas n'ayant aucun ordre ni rangement possible, la seule façon garantie de trouver la clé consiste à soulever chaque feuille une par une, de la première jusqu'à la dernière.*

2. La Recherche Dichotomique (Principe du Diviser pour Régner)

La recherche dichotomique exploite la propriété d'ordre d'un tableau trié pour éliminer de grands ensembles d'éléments sans même les examiner. L'algorithme délimite la fenêtre de recherche active grâce à deux pointeurs d'indices : 'debut' (initialisé à 0) et 'fin' (initialisé à n-1). À chaque itération de la boucle de recherche, l'algorithme calcule l'indice central par division entière : milieu = (debut + fin) / 2. Il compare ensuite la valeur T[milieu] avec la cible recherchée : 1. Si T[milieu] == cible : L'élément est trouvé, la recherche s'arrête (succès). 2. Si cible < T[milieu] : L'élément ne peut se trouver que dans la moitié gauche. On restreint la zone en déplaçant le pointeur de fin : fin = milieu - 1. 3. Si cible > T[milieu] : L'élément ne peut se trouver que dans la moitié droite. On déplace le pointeur de début : debut = milieu + 1. Le processus s'itère tant que debut <= fin et que l'élément n'a pas été trouvé. En divisant l'espace de recherche par deux à chaque étape, l'algorithme atteint une complexité logarithmique O(log2 n). Pour un tableau de 1 000 000 d'éléments triés, 20 comparaisons suffisent au maximum (car 2^20 ≈ 1 048 576), contre 1 000 000 pour la recherche séquentielle.

• Précondition stricte et obligatoire : le tableau doit être préalablement et strictement trié.

• Paradigme algorithmique Diviser pour Régner : réduction de 50 % de l'espace de recherche à chaque étape.

• Gestion dynamique de 3 pointeurs d'indices : debut, fin, et milieu = (debut + fin) / 2.

• Complexité temporelle logarithmique O(log2 n) au pire des cas (ex: 20 comparaisons max pour 1 000 000 d'éléments).

Cas concret enseignant : *Exemple du professeur : Le jeu du 'Devine un nombre entre 1 et 100'. Proposer 1, 2, 3... est une stratégie séquentielle inefficace. Proposer 50 élimine d'un coup 50 possibilités selon que la réponse est 'c'est plus grand' ou 'c'est plus petit'. En réitérant cette dichotomie, la réponse est garantie en 7 étapes maximum (car 2^7 = 128 > 100).*

3. Arbitrage Algorithmique et Analyse de Performance

En algorithmique, le choix d'une méthode ne repose pas uniquement sur sa vitesse intrinsèque dans le cas idéal, mais sur l'évaluation du contexte global du problème. La recherche dichotomique offre une performance remarquable en O(log n), mais exige la précondition d'un tableau trié. Or, trier un tableau désordonné nécessite au mieux O(n log n) opérations (via des algorithmes comme le Tri Fusion ou le Tri Rapide). Trois cas de figure guident l'arbitrage : 1. Recherche ponctuelle (unique) sur un tableau non trié : Exécuter directement une recherche séquentielle O(n) est plus rapide que de trier d'abord le tableau puis de chercher (O(n log n + log n) > O(n)). 2. Tableau déjà trié : La recherche dichotomique en O(log n) s'impose comme le choix optimal sans réserve. 3. Tableau statique soumis à de très nombreuses requêtes (k recherches) : Réaliser le tri initial en O(n log n) une fois pour toutes est hautement rentable, car le coût total O(n log n + k * log n) devint très rapidement inférieur à k * O(n) lorsque k augmente.

• Coût algorithmique du tri préalable : trier un tableau désordonné exige une complexité de O(n log n).

• Recherche unique sur tableau non trié : la recherche séquentielle O(n) est plus rapide que Trier + Recherche Dichotomique.

• Recherches multiples sur tableau statique : l'investissement initial du tri en O(n log n) est largement amorti par k recherches dichotomiques en O(log n).

Cas concret enseignant : *Analogie du professeur : Le dictionnaire imprimé. Ordonner l'intégralité des mots du dictionnaire par ordre alphabétique demande un travail considérable à l'imprimeur (le tri initial), mais cet effort unique permet ensuite à des millions d'utilisateurs de retrouver n'importe quel mot en quelques secondes pendant des années.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique puisqu'elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : Parce que la recherche dichotomique exige une précondition stricte : le tableau doit être trié au préalable. Si le tableau n'est pas trié, le coût du tri initial est en O(n log n). Pour effectuer une seule recherche rapide sur des données non triées, il est plus économique de faire une recherche séquentielle en O(n) que de trier d'abord puis de chercher en dichotomie.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière : milieu = (debut + fin) / 2. La division entière tronque automatiquement la partie décimale. Par exemple, si la zone s'étend de l'indice 0 à 9 (10 éléments), milieu = (0 + 9) / 2 = 4. L'algorithme prend la case d'indice 4 comme élément pivot milieu, ce qui découpe le tableau de façon valide sans bloquer l'algorithme.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié d'entiers T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices de 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour localiser la valeur cible 42. Rédiger ensuite la fonction C++ correspondante en respectant la norme d'apprentissage de l'IUT d'Anglet.

Démarche de résolution :
Trace d'exécution pas à pas pour la recherche de valRecherchee = 42 dans T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] : • Initialisation : - debut = 0 - fin = 9 (indice du dernier élément : 10 - 1) - pos = -1 (symbole d'absence) • Itération 1 : - Calcul du milieu : milieu = (0 + 9) / 2 = 4 (division entière). - Valeur examinée : T[4] = 19. - Comparaison : Cible (42) > T[4] (19). - Décision : La valeur cible est strictement supérieure. La recherche se poursuit dans la partie droite. - Mise à jour du pointeur debut : debut = milieu + 1 = 4 + 1 = 5. • Itération 2 : - Fenêtre de recherche active : debut = 5, fin = 9. - Calcul du milieu : milieu = (5 + 9) / 2 = 7. - Valeur examinée : T[7] = 42. - Comparaison : Cible (42) == T[7] (42). - Décision : Correspondance exacte localisée ! pos = 7. • Arrêt de la boucle : La condition pos == -1 devient fausse.

```
/*
  En-tete de fichier
  Auteur : Matei
  Matiere : R1.01 - Algorithmique Fondamentale
  Date : 13/11/2026
  Objectif : Algorithme de recherche dichotomique dans un tableau d'entiers trie
*/

#include <iostream>
#include <vector>

using namespace std;

int main()
{
    // CONSTANTES
    const int TAILLE = 10;
    const int CIBLE = 42;

    // VARIABLES
    vector<int> t = {3, 8, 12, 15, 19, 24, 31, 42, 50, 61};
    int debut;
    int fin;
    int milieu;
    int pos;

    // TRAITEMENTS
    debut = 0;
    fin = TAILLE - 1;
    pos = -1;

    while ((debut <= fin) && (pos == -1))
    {
        milieu = (debut + fin) / 2;
        if (t[milieu] == CIBLE)
        {
            pos = milieu;
        }
        else if (CIBLE < t[milieu])
        {
            fin = milieu - 1;
        }
        else
        {
            debut = milieu + 1;
        }
    }

    if (pos != -1)
    {
        cout << "Valeur " << CIBLE << " trouvee a l'indice : " << pos << endl;
    }
    else
    {
        cout << "Valeur non trouvee dans le tableau." << endl;
    }

    return 0;
}
```

Résultat : La valeur 42 a été localisée à l'indice 7 du tableau en seulement 2 comparaisons grâce à la recherche dichotomique. En comparaison, une recherche séquentielle classique aurait demandé 8

comparaisons successives (de T[0]=3 jusqu'à T[7]=42).

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour pouvoir appliquer une recherche dichotomique sur un tableau ?

R : Le tableau doit obligatoirement être strictement trié au préalable (par exemple par ordre croissant).

Q : Quelle est la complexité temporelle au pire des cas de la recherche séquentielle et de la recherche dichotomique ?

R : La recherche séquentielle a une complexité linéaire O(n) au pire des cas, tandis que la recherche dichotomique a une complexité logarithmique O(log2 n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser un élément dans un tableau trié de 1024 éléments ?

R : Elle nécessite au maximum 10 comparaisons, car log2(1024) = 10 (puisque 2^10 = 1024).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765